1 Open

CI/CD Capabilities & Rules Reference

> **This is the single source of truth for testing & CI in this repo.** The tables between the `<!-- BEGIN/END:GENERATED -->` markers are **auto-generated** by `ci/gen_capabilities_doc.py` from `.gitlab-ci.yml` (jobs/rules), the model/dataset code (capability matrix), and `ci/policy.yml` (rationale) — **do not edit them by hand**. Regenerate with `make docs-ci`. On the default branch the `docs:autogen` job regenerates and commits this file automatically, so the tables never go stale. The prose sections (including the Developer guide below) are hand-written and preserved across regeneration. For **how the auto-generation works** (sources of truth, the marker system, the `docs:check` / `docs:autogen` jobs, how to add a table), see `how-ci-cd-docs-are-generated.md`.
>
> **See also:** `future_pydantic_tests.md` (planned Pydantic validation tests) and `MODELZOO_ARCHITECTURE_NOTES.md` (packaging / serving notes). The former developer guide and CI plans were merged into this file.

Purpose

This document is the single reference for **what the CI/CD pipeline tests, and under which rules**, so it can be shared with colleagues. To request a change to any rule, gate, or capability, open an issue with the **CI Change Request** template (`.gitlab/issue_templates/CI_Change_Request.md`).

Pipeline stages

Stage order: `sanity` → `validate` → `lint` → `test` → `coverage` → `docs` → `build` → `test-wheel`

Jobs, triggers, and gates

Every job, the condition under which it runs, and whether a failure blocks the pipeline or only warns.

Stage	Job	Trigger	On failure	Runs
sanity	`sanity:check-secrets`	`always`	Blocks	./ci/check_secrets.sh .
sanity	`sanity:check-structure`	`always`	Blocks	test -f pyproject.toml
sanity	`sanity:python-syntax`	`always`	Blocks	find seanergys_modelzoo ci tests scripts -name "*.py" -print0 | xargs -0 -r python -m py...
validate	`validate:yaml-json`	`always`	Blocks	python ci/validate_yaml.py
lint	`lint:format`	`rules_code`	Warns	ruff format --check seanergys_modelzoo ci tests
lint	`lint:mypy`	`rules_code`	Warns	mypy seanergys_modelzoo ci tests --ignore-missing-imports --explicit-package-bases
lint	`lint:python`	`rules_code`	Warns	ruff check seanergys_modelzoo ci tests
test	`test:audit`	`rules_deps`	Warns	poetry export -f requirements.txt --without-hashes --with dev,ci -o /tmp/poetry-audit-req...
test	`test:integration`	`rules_integration`	Blocks	CI_SKIP_ZENODO=1 poetry run pytest tests/integration/ -v -m "not zenodo" --cov=seanergys_...
test	`test:smoke-data`	`rules_smoke`	Blocks	poetry run pytest tests/smoke/test_smoke_data.py -v -m "not zenodo" --cov=seanergys_model...
test	`test:smoke-models`	`rules_smoke`	Blocks	poetry run pytest tests/smoke/test_smoke_models.py -v --cov=seanergys_modelzoo --cov-repo...
test	`test:unit`	`rules_unit`	Blocks	poetry run pytest tests/unit/ -v --tb=short --cov=seanergys_modelzoo --cov-report=
test	`test:zenodo`	`rules_zenodo`	Warns	poetry run pytest tests/integration/ -v -m zenodo
coverage	`coverage:report`	`rules_code`	Blocks	if ls coverage-data/.coverage.* >/dev/null 2>&1 (closed); then coverage combine coverage-data cove...
docs	`docs:autogen`	`default-branch`	Warns	poetry run python ci/gen_capabilities_doc.py
docs	`docs:check`	`merge-request`	Warns	poetry run python ci/gen_capabilities_doc.py --check
build	`build:wheel`	`rules_wheel`	Blocks	poetry build --format wheel
test-wheel	`test:wheel`	`rules_wheel`	Blocks	python ci/verify_wheel.py

Trigger legend

Trigger	When the job runs
`always`	Runs on every pipeline (cheap checks < 10s, no change filter).
`default-branch`	Runs only on the default branch (post-merge).
`merge-request`	Runs on branches / merge requests (excluded on the default branch).
`rules_code`	Any source, test, script, CI, or dependency change.
`rules_deps`	Only pyproject.toml / poetry.lock / .gitlab-ci.yml changed.
`rules_integration`	Source, tests/integration/**, conftest, create_sample_data.py, or deps.
`rules_smoke`	Source, tests/smoke/**, conftest, or dependency change.
`rules_unit`	Source, tests/unit/**, conftest, or dependency change.
`rules_wheel`	Source or pyproject.toml changed (packaging-relevant).
`rules_zenodo`	Default branch, or a change under seanergys_modelzoo/datasets/**, tests/integration/**, or conftest.

Pass / fail engineering decisions

Why each job blocks the pipeline or is allowed to fail. The *what* (above) is auto-derived from `.gitlab-ci.yml`; the *why* below is maintained in `ci/policy.yml`.

Job	On failure	Why
`sanity:check-secrets`	Blocks	Warn-only: the regex scan can false-positive; a human triages hits.
`sanity:check-structure`	Blocks	Guards the repo layout that every later job and import assumes.
`sanity:python-syntax`	Blocks	Broken syntax must never enter the pipeline; a cheap hard gate.

`validate:yaml-json`	Blocks	Invalid YAML/JSON (incl. this pipeline) would break CI itself — hard gate.
`lint:format`	Warns	Formatting drift shouldn't block merges; run `make format` to fix.
`lint:mypy`	Warns	Type coverage is incomplete; surfaced as a warning, not a gate.
`lint:python`	Warns	Style/lint issues are quality signals, not correctness; non-blocking.
`test:audit`	Warns	Dependency CVEs are informational and often unfixable immediately; warn-only.
`test:integration`	Blocks	End-to-end model×dataset pipeline must pass; the core correctness gate.
`test:smoke-data`	Blocks	Dataset contract + fixtures must load; hard gate (runs without Zenodo).
`test:smoke-models`	Blocks	Model discovery/instantiation must hold or nothing downstream works.
`test:unit`	Blocks	Component correctness is a hard gate — the fastest signal of breakage.
`test:zenodo`	Warns	The only job that reaches Zenodo: downloads a small is_dummy subset (memory-safe capped read — FDataDataset.max_rows via pyarrow.iter_batches over an fsspec HTTP range-request stream, ~136 MB not the ~2 GB whole row group) and runs train→predict→save/load. Warn-only because Zenodo is an external service — downtime must not block merges; a failure is a visible connectivity/regression signal. Flip allow_failure to false to make it a hard gate.
`coverage:report`	Blocks	Combines per-job coverage into one report; fails only on tooling error.
`docs:autogen`	Warns	Regenerates + commits docs on the default branch; warn-only so a token or push hiccup never breaks main.
`docs:check`	Warns	Warns on merge requests when docs/ci-cd.md is stale; not blocking.
`build:wheel`	Blocks	A non-buildable package is a release blocker — hard gate.
`test:wheel`	Blocks	The distributed wheel must import cleanly for consumers — hard gate.

Model × dataset capability matrix

Which models are tested against which datasets. Discovered automatically from each model's `supported_datasets` and the auto-discovery in `tests/conftest.py` — adding a model or dataset updates this table on the next regeneration.

Model	Task	Supported dataset	DATASET_ID	Tested in CI
ColocationPredictor	Regression	ColocationDataset	`colocation`	yes
DurationPredictionBull	Regression	FDataDataset	`f-data`	yes
JPCP	Regression	PM100Dataset	`pm100`	yes
JPCP	Regression	FDataDataset	`f-data`	yes
MACK	Classification	FDataDataset	`f-data`	yes
MCBound	Classification	FDataDataset	`f-data`	yes
RUAD	Classification	M100Dataset	`–`	no

Legend — the "Tested in CI" column:

- `yes` — the full train → predict → save → load pipeline runs for this pair and **must pass** (a hard gate).
- `no` — the pair produces no test case at all, because the model cannot be built or the dataset declares no `DATASET_ID`. That is itself a failure (`test_every_supported_pair_is_exercised_or_declared`), not a quiet state.

There is **no `xfail` state**. The lists that produced one (`integration_xfail`, `model_xfail`, `config_load_xfail`) were removed on 2026-09-13 — see No expected failures. A pair that is exercised and does not work is a **failing test** and a red pipeline.

> **Ownership.** `ci/`, `tests/`, `.gitlab-ci.yml`, `Makefile`, `scripts/` and `docs/` are maintained by the CI/test maintainer. Everything under `seanergys_modelzoo/` — models, datasets, common base classes — belongs to the model authors. When a test fails because of core code, CI's job is to **report it precisely**, not to fix it and not to hide it.

Datasets

Dataset	DATASET_ID	is_dummy mode	Exercised by CI
ColocationDataset	`colocation`	yes	yes
FDataDataset	`f-data`	yes	yes
M100Dataset	`–`	yes	yes
PM100Dataset	`pm100`	yes	yes
SCRIPTAIDataset	`–`	yes	no
ScalingDataset	`scaling`	yes	yes

Discovered but not exercised — the coverage gaps

The modelzoo's promise is that a developer adds a model or dataset and CI tests it **with no test edit**. Auto-discovery finds the new class; what makes the promise real is that something *fails* when a discovered class is never actually run. `tests/unit/test_discovery_contract.py` does that: a dataset that no model consumes fails `test:unit` unless `ci/policy.yml -> dataset_scope` records why, what is missing, and who fixes it.

That escape hatch is deliberately a **CI-side record, not a change to the dataset**. The modelzoo core belongs to the model and dataset authors; CI publishes the gap and names the owner's fix rather than silently working around it. An empty table below means every discovered dataset is genuinely exercised.

Dataset	What is missing	Why it is not tested	Who closes it, and how
SCRIPTAIDataset	DATASET_ID (inherits the empty default) and no model lists it in supported_datasets.	SCRIPT-AI is discovered but no model consumes it, so it contributes no test cases. It is served from GitHub rather than Zenodo, and loading it fetches one job-script file per 'jid' in features.csv, so a single sample parquet cannot represent the source either.	The dataset author sets DATASET_ID = "script-ai" and a model declares SCRIPTAIDataset in supported_datasets with a matching get_train_config entry.

Core change requests

Changes needed inside `seanergys_modelzoo/` that **CI does not make itself**.

The core belongs to the model and dataset authors; the CI/test layer never edits it. When a test cannot be written correctly because of something in core, the request is recorded in `ci/policy.yml -> core_change_requests`, published here, and handed to the owner. CI reports the truth about the code — it does not reshape the code to make itself pass.

Each entry carries a machine-checkable condition, so `tests/unit/test_core_change_requests.py` fails in **both** directions: a vague entry fails, and an entry whose fix has already landed fails too, forcing its removal. A request is therefore either open and accurate, or deleted — it cannot sit here making a false claim about the code.

ID	Tracker	Symbol	Owner	What must change	Why CI cannot do it
`CR-1`	#12 (closed)	`M100Dataset`	M100 dataset author	Set `DATASET_ID: ClassVar[str] = "m100"` on M100Dataset AND change RUAD.get_train_config / get_test_config to accept that id: they currently key on the invented literal "M100", so even after DATASET_ID is set the call raises ValueError("RUAD only supports M100"). Both halves are needed or the pair still produces no test case.	Setting it from the test layer would mean CI editing a dataset it does not own, and inventing an id and a model pairing on the author's behalf. CI can only report that the contract is unmet.
`CR-2`	#18	`SCRIPTAIDataset`	SCRIPT-AI dataset author	Set `DATASET_ID: ClassVar[str] = "script-ai"` on SCRIPTAIDataset, and have a model list SCRIPTAIDataset in supported_datasets with a matching get_train_config entry.	Same as CR-1 — the id and the model pairing are the author's decision, not CI's.
`CR-3`	#17 (closed)	`FDataDataset.get_statistics_by_partition`	F-DATA dataset author	Rewrite get_statistics_by_partition() against F-DATA's real published columns, or remove it. It currently reads 'partition', 'n_nodes' and 'power_consumption'; the real columns are 'nnumr'/'nnuma' (nodes), 'avgpcon' (average power) and 'duration', and raw F-DATA has no partition column at all.	Choosing the replacement grouping key (or deciding to drop the method) is a dataset-semantics decision owned by the author. CI must not silently rewrite a public method to make a test pass.
`CR-4`	#11 (closed)	`RUAD.model_post_init`	RUAD model author	Do not construct M100Dataset inside RUAD.model_post_init(). It is built there only to read one sample and derive _input_dim; make the input dimension an explicit field (or defer building the network to the first training batch) so constructing the model is pure. get_train_config() already returns the dataset CLASS + params, matching JPCP/MACK/MCBound — the constructor should do the same.	The fix is a restructuring of the model's own initialisation and its data-loading contract; only the model author can decide where the dataset should be built.
`CR-5`	#13 (closed)	`MACK.get_train_config.preprocess_fdata`	MACK model author	Stop overwriting the real label column: preprocess_fdata() assigns the KMeans result to df["pclass"], which is F-DATA's published memory/compute-bound label. Write the derived class to its own column (e.g. kmeans_pclass) and make the value a scalar class id rather than a (flops_class, mbw_class) tuple. Keep pclass as the target and label-encode it to integer ids for XGBoost.	Which column carries the target, and whether the KMeans class replaces or complements it, is a modelling decision owned by the author. CI can only report that the pair cannot run.
`CR-6`	#15 (closed)	`PM100Dataset.validate_model (pm100.py:71-74) / JPCP.get_train_config`	PM100 dataset author (primary) and JPCP model author	Let a model declare DERIVED input features. PM100Dataset.validate_model() unions input_features + output_features into `columns` even when the author passed an explicit `columns` list, and that union becomes the parquet read projection — so a feature created by preprocessing can never be named. Preferred fix (dataset): when `columns` is given, use it as the read projection and validate input/output features against the frame AFTER preprocessing_functions have run. Alternative (model): JPCP stops naming num_nodes_req_cat / user_id_cat in input_features and selects them after preprocessing.	The projection rule is part of the dataset's public contract and affects every model that uses it; and which columns JPCP should feed its encoder is the model author's design decision. CI must not change either.
`CR-7`	#14 (closed)	`MCBound.get_train_config`	MCBound model author	Encode the target. The config trains on output_features=['pclass'], which F-DATA publishes as a STRING ('memory-bound', ...). Add a preprocessing_functions entry that label-encodes it to integer ids, and persist the encoder in save()/load() so get_train_config and get_test_config agree on the class ids.	The encoding scheme and where the fitted encoder is persisted are part of the model's own contract; a test-side encoder would not survive save()/load() and would misrepresent what the shipped model does.
`CR-8`	#16 (closed)	`SeanergysParquetDataset.load_data / preprocessing_functions`	Dataset base-class author	Make preprocessing_functions mean ONE thing. The base class treats each entry as a (columns, fn) tuple and calls fn(df[columns]); PM100Dataset and FDataDataset treat each entry as a plain callable and call fn(df). Pick one contract — the DataFrame-in/DataFrame-out callable used by both concrete datasets is the more capable one, since it can add derived columns — and document it on the field in SeanergysDataset.	Choosing the surviving contract changes the public API of every dataset and would silently break any caller written against the other form; that is the dataset authors' decision, not CI's.

Test inventory

Every test file grouped by layer, with the check it performs (from its docstring).

Layer	File	Checks
unit	`test_configurator.py`	Unit tests for Seanergys configurator (Pydantic params).
unit	`test_dataloader.py`	Unit tests for SeanergysDataloader (type guard + from_config).
unit	`test_dataloader_params.py`	Unit tests for SeanergysDataloaderParams validation.
unit	`test_logger.py`	Unit tests for SeanergysLogger.
unit	`test_metadata.py`	Unit tests for SeanergysModelMetadata.
unit	`test_sklearn_model.py`	Unit tests for SeanergysSklearnModel.
unit	`test_utils.py`	Unit tests for ci.utils (discovery helpers).
smoke	`test_smoke_data.py`	Smoke tests: dataset discovery, contract validation, and data loading.
smoke	`test_smoke_models.py`	Smoke tests: model discovery, instantiation, and train-config structure.
integration	`test_model_pipeline.py`	Integration tests: the full pipeline for every (model, dataset) pair.

Test-skip inventory

A skipped test is the one place a **green** pipeline can silently hide an untested path, so every skip in the suite is inventoried below: where it is, why it skips, and exactly how to make it run for real. This table is regenerated from the source (the `pytest.skip(...)` calls and `@pytest.mark.zenodo` markers are scanned via AST; implicit skips come from `ci/policy.yml` `skip_notes`), so it can never drift from the code.

File	Test (line)	Reason	How to remove the skip
`tests/integration/test_model_pipeline.py`	`test_full_pipeline_zenodo` (L178)	Needs a live Zenodo download; deselected by `-m "not zenodo"` and skipped when CI_SKIP_ZENODO=1.	Runs for real in the `test:zenodo` job.

`tests/integration/test_model_pipeline.py`	`test_full_pipeline_zenodo` (L183)	CI_SKIP_ZENODO=1 — Zenodo tests disabled for this run.	Runs for real in the `test:zenodo` job (no CI_SKIP_ZENODO); locally run with network.
`tests/integration/test_model_pipeline.py`	`test_full_pipeline_zenodo` (L196)	{...}/{...}: is_dummy=True returned 0 rows	Implement dummy support in {...}.
`tests/smoke/test_smoke_data.py`	`test_dataset_fixture_has_data` (L192)	fixture not found: {...}	Run `make sample-data` to create it.
`tests/smoke/test_smoke_data.py`	`test_dataset_loads_from_zenodo` (L262)	Needs a live Zenodo download; deselected by `-m "not zenodo"` and skipped when CI_SKIP_ZENODO=1.	Runs for real in the `test:zenodo` job.
`tests/smoke/test_smoke_data.py`	`test_dataset_loads_from_zenodo` (L279)	CI_SKIP_ZENODO=1 — Zenodo tests disabled for this run.	Runs for real in the `test:zenodo` job (no CI_SKIP_ZENODO); locally run with network.
`tests/smoke/test_smoke_data.py`	`test_dataset_loads_from_zenodo` (L290)	use_zenodo_url=True not supported.	Add use_zenodo_url field to {...} to enable Zenodo testing.
`tests/smoke/test_smoke_data.py`	`test_dataset_loads_with_is_dummy` (L210)	Needs a live Zenodo download; deselected by `-m "not zenodo"` and skipped when CI_SKIP_ZENODO=1.	Runs for real in the `test:zenodo` job.
`tests/smoke/test_smoke_data.py`	`test_dataset_loads_with_is_dummy` (L229)	CI_SKIP_ZENODO=1 — Zenodo tests disabled for this run.	Runs for real in the `test:zenodo` job (no CI_SKIP_ZENODO); locally run with network.
`tests/smoke/test_smoke_models.py`	`test_model_get_train_config` (L122)	get_train_config() returned None.	Implement get_train_config() in {...} to enable this test.
`tests/unit/generic/test_core_change_requests.py`	`test_request_is_still_open` (L136)	target {...} is not present on this branch, so the request cannot be evaluated he...	{...}
`tests/unit/generic/test_discovery_contract.py`	`test_config_loads_offline_from_committed_fixture` (L256)	no committed fixture {...}.	Run `make sample-data` to create it.
`tests/unit/generic/test_fixture_provisioning.py`	`test_no_undeclared_fixture_gap_when_downloading_is_disabled` (L290)	the fixture download failed, so this cannot be judged here: {...}	Re-run when the source is reachable, or commit the fixture.
`tests/unit/generic/test_model_lifecycle.py`	`test_save_returns_false_on_write_error` (L123)	{...} does not persist via joblib	—
`tests/unit/specific/test_scaling_predictor_functional.py`	`test_zenodo_dataset_has_every_candidate_feature_and_target` (L132)	Needs a live Zenodo download; deselected by `-m "not zenodo"` and skipped when CI_SKIP_ZENODO=1.	Runs for real in the `test:zenodo` job.

Developer guide

Everything below is hand-written prose (preserved across regeneration). It is the merged, updated content of the former `TESTING_AND_CI_GUIDE.md` and the old CI plans.

Setting up a machine

The only things you must have are `curl`, `git` and `make`. Everything else is installed for you on first use, into `~/.local/bin` and `.venv/`.

```
make test          # on a fresh clone this bootstraps the machine, then runs the tests
```

Every target that needs a working environment depends on `make env`, which chains three guarded steps and skips any of them that is already satisfied:

Step	Target	What it guarantees
1	`make tools`	`curl` / `git` / `make` present; **uv** and **poetry** installed if missing; a Python in `>=3.12,<3.15` (fetched with `uv python install` if the system one is out of range). Runs `scripts/ensure_tools.sh`.
2	`make deps-check`	`poetry.lock` is in sync with `pyproject.toml` (`poetry check --lock`)
3	`$(VENV)/.deps-installed`	`.venv/` exists and `poetry install --with dev,ci` has run. The stamp file depends on `pyproject.toml` and `poetry.lock`, so the install re-runs **only** when one of them actually changed.

Useful setup targets:

Command	Does
`make bootstrap`	The whole setup explicitly, without running tests
`make doctor`	Reports toolchain / `.venv` / fixture state and changes nothing
`make tools`	Toolchain only — installs what is missing
`make tools-check`	Toolchain only — **never** installs, non-zero exit if incomplete
`make env`	Toolchain + lock check + `.venv` synced to `poetry.lock`
`make install`	`dev` group only (tooling). Not enough to run the tests — they import torch/sklearn through the package — so it clears the stamp and the next test target reinstalls `dev,ci`.
`make install-ci`	`dev` + `ci` groups (same as `make env`)
`make clean-venv`	Deletes `.venv/`; the next make target rebuilds it

Environment switches:

Variable	Effect
`MODELZOO_NO_AUTO_INSTALL=1`	Never install machine tools — report what is missing and fail. Use on locked-down machines and in images that must not fetch anything.

`PYTHON_BIN=/usr/bin/python3.13`	Force the interpreter used to create `.venv/`
`PYTHON_KEYRING_BACKEND=…`	Defaults to the **null** backend. Poetry otherwise queries the system keyring and aborts mid-install on sessions without an unlocked DBus secret service (`DBusErrorResponse` → `Cannot install <pkg>`). Override only if you add a private package index.
`CI_SKIP_ZENODO=1`	Never touch the network; offline fixtures only

If a fresh shell cannot find `uv` / `poetry` afterwards, add `export PATH="$HOME/.local/bin:$PATH"` to your `~/.bashrc`.

Running the tests locally

Run the tests before every push — it is faster than waiting for a red pipeline. No setup command is required first; each of these bootstraps what it needs.

```
make sample-data    # optional — refreshes the Zenodo fixtures (they are committed already)
```

Command	Runs	When
`make test`	unit + smoke (offline, fast)	**before every push**
`make test-unit`	`tests/unit/`	changing base classes / utils
`make test-smoke-models`	`tests/smoke/test_smoke_models.py`	model discovery / instantiation
`make test-smoke-data`	`tests/smoke/test_smoke_data.py`	dataset contract / fixtures
`make test-pipeline`	full train→predict→save→load per model	changing model code
`make test-integration`	`tests/integration/` (offline, fixtures)	before opening an MR
`make test-all`	unit + smoke + integration	final check before an MR
`make test-wheel`	build the wheel and verify it imports	packaging changes
`make lint` / `make format` / `make typecheck`	ruff check / ruff format / mypy	quality gates
`make docs-ci` / `make docs-ci-check`	regenerate / drift-check this file	after CI or test changes
`make pre-push`	format-check + check-secrets + lint + test	the full local gate
`make ci`	mirrors the whole GitLab pipeline offline	simulate CI locally

Skip Zenodo network locally (uses committed fixtures only, no downloads):

```
CI_SKIP_ZENODO=1 make test-integration
```

Run a single test file or test:

```
.venv/bin/pytest tests/unit/test_configurator.py -v
.venv/bin/pytest tests/smoke/test_smoke_models.py::test_model_discovery -v
```

What each test layer checks

- **unit** (`tests/unit/`) — component correctness in isolation: Pydantic params, base-class contracts, dataset parsers, per-class dataset internals (parsing, node ranges), the sklearn wrapper. Hard gate.
- **smoke** (`tests/smoke/`) — cheap end-to-end sanity without training: every model and dataset is **discovered** (`tests/conftest.py`), instantiates, and its `get_train_config` has the right shape; fixtures load. Hard gate, runs offline.
- **integration** (`tests/integration/test_model_pipeline.py`) — the full train→predict→save→load lifecycle for **every discovered (model, dataset) pair**, offline against committed fixtures. Any failure names the exact pair **and** stage. Hard gate.
- **zenodo** (`test:zenodo` , `-m zenodo`) — the same lifecycle against a real `is_dummy=True` download from Zenodo. The **only** job that touches the network; warn-only.

Test fixtures — provisioned automatically

The suite fetches its own data. There is no step a contributor has to remember.

`tests/fixture_provisioning.py` runs from `pytest_sessionstart` (wired in `tests/conftest.py`): before any test executes, every **discovered** dataset is checked for its sample parquet in `tests/fixtures/sample_data/` , and anything missing is downloaded as a small `is_dummy` -sized chunk via `scripts/create_sample_data.py` . A fixture already on disk is never re-fetched, so the normal cost is a handful of `stat()` calls.

The same check exists on the make side (`make ensure-fixtures` , a dependency of every data-driven target). Both call `create_sample_data.missing_fixtures()` , and `tests/unit/generic/test_fixture_provisioning.py` fails if the two ever disagree — so `make test` and a bare `pytest` can never differ on whether the suite has its data.

Environment	Behaviour
(nothing set)	`auto` — download only what is missing
`MODELZOO_FIXTURES=never`	never download; report the gap and let data tests skip
`MODELZOO_FIXTURES=force`	re-download every fixture, even if present
`CI_SKIP_ZENODO=1`	implies `never` — that flag means "this run does not touch the network"
`MODELZOO_FIXTURE_TIMEOUT`	seconds before a stuck download is abandoned (default 1800)

CI sets `MODELZOO_FIXTURES: "never"` **globally** in `.gitlab-ci.yml` . The fixtures are committed, and fetching one on a runner pulls a multi-hundred-MB Zenodo file into memory, which OOM-kills the job (exit 137). So on a runner a missing fixture is a missing *commit*, and `test_no_undeclared_fixture_gap_when_downloading_is_disabled` **fails** the unit job with that message rather than letting the dataset skip its way to a green pipeline. The escape hatch is the same as everywhere else: record the dataset under `dataset_scope` in `ci/policy.yml` with the reason it cannot be sampled.

A download that is attempted and fails (an outage at the source) is reported as a SKIP, not a failure — an external service being down must not turn an unrelated run red.

No hardcoded model or dataset names (enforced)

Auto-discovery only delivers its promise — *add a model or dataset, CI exercises it, no test edit* — while nothing in the machinery keys its behaviour on a concrete name. Two tests enforce that, and both take the names to hunt for **from discovery itself**, so they need no maintenance:

Test	Scans	Rejects
`tests/unit/generic/test_no_hardcoded_names.py`	`tests/unit/generic/`	any mention of a discovered model/dataset class
`tests/unit/generic/test_no_hardcoded_names_repo.py`	`.gitlab-ci.yml` , `Makefile` , `ci/**/*.py` , `scripts/**/*.py` , `tests/**/*.py`	class names, `DATASET_ID` values and `sample_*.parquet` names **in executable content**

The repo-wide scan separates code from prose: Python is parsed with `ast` (comments are invisible, docstrings skipped) and YAML/Makefile are scanned with comment lines stripped. `FDataDataset -> sample_fdata.parquet` in a docstring documents the rule and keeps working when a dataset is added; `if name == "FDataDataset":` does not.

Where a name IS allowed, and why:

- `tests/unit/specific/` — component tests, where naming the component is the point.
- `ci/policy.yml` — statements *about* a component (`dataset_scope`, `core_change_requests`).
- `archive/` — retired material, run by nothing. See `archive/README.md`.
- prose anywhere.

The surfaces and the allowlist live in `ci/policy.yml → name_scan`, not in the test, so widening or narrowing the scan is a reviewable policy change — and every exemption must carry a reason and still match a real file.

Every model, on every dataset it claims

`supported_datasets` is the model author's promise about what the model works with, and `test_every_supported_pair_is_exercised_or_declared` holds them to it: for each discovered model, **every** declared dataset must yield a usable configuration from `get_train_config(<DATASET_ID>)`. Previously a `get_train_config` that raised for one of the model's datasets simply dropped that pair from discovery — the model could advertise three datasets, be exercised on one, and CI would still report green. Now the gap either gets fixed or gets recorded in `ci/policy.yml` (`core_change_requests` / `dataset_scope`) and published in the tables above — recording it documents the defect, it does not make the test pass.

`is_dummy=True` and memory-safe Zenodo loading

`is_dummy=True` gives each dataset a tiny **real** slice of its data for testing. For F-DATA it sets `files="21_04"` and a small **row cap** (`max_rows=200`): the read goes through `FDataDataset._read_parquet`, which uses `pyarrow.ParquetFile.iter_batches` over an `fsspec` HTTP **range-request** stream and stops after the first 200 rows. This matters because F-DATA's monthly files are a **single row group**, so a `filters=` predicate cannot push down — a plain `pd.read_parquet(url, columns=, filters=)` would still materialise the whole group (~2 GB). The capped read fetches only the selected columns' chunks (~136 MB) and never OOMs the runner. `max_rows=None` (the default) restores the plain read for local files. This needs `aiohttp` + `fsspec` (declared in the `ci` dependency group).

Adding a new model or dataset

The suite is **auto-discovery driven**, so a new model or dataset enters CI by contract — no test or table to edit by hand.

1. Create `seanergys_modelzoo/models/tasks/<task>/<model_name>/<model_name>_model.py`.
2. Subclass `SeanergysSklearnModel` (or `SeanergysModel` for non-sklearn).
3. Implement `get_train_config(dataset_id)` returning `Dict[str, Tuple[model_instance, DatasetClass, (SeanergysDatasetParams, SeanergysDataloaderParams)]]`, and raise on an unknown `dataset_id` (see `JPCP` for the pattern).
4. Ensure each dataset it uses supports `is_dummy=True` (CI exercises this without real data).
5. `tests/conftest.py` discovers the model, adds its `(model, dataset)` pairs to `MODEL_DATASET_PAIRS`, and the smoke/integration/capability tables pick it up on the next run. Regenerate this doc with `make docs-ci`.

If a (model, dataset) pair is known-broken, it stays **red**. Record the fix under `core_change_requests` in `ci/policy.yml` so the owner, the evidence and the expiry condition are published — there is no xfail list and no way to mark a failure as acceptable.

Common workflows

- **Fresh clone / new machine:** nothing — `make test` sets everything up. `make doctor` if you want to see the state first.
- **Before every push:** `make pre-push` (format-check + secrets + lint + unit/smoke).
- **Before opening an MR:** `make test-all` and `make docs-ci-check`.
- **After changing `.gitlab-ci.yml`, `ci/policy.yml`, or test structure:** `make docs-ci` to regenerate the tables above, and commit the result.
- **To request a CI change (colleagues):** open an issue with the **CI Change Request** template (`.gitlab/issue_templates/CI_Change_Request.md`).

Tooling

`pytest` (+ `pytest-cov`) for tests; `ruff` for lint **and** formatting (Black is not used in CI); `mypy` for types; `pip-audit` for dependency CVEs (see `ci/policy.yml` `audit_ignore`); `py_compile` for the syntax gate; `PyYAML` for the doc generator and validators. Dependencies are managed with Poetry groups: `dev` (tooling) and `ci` (the full ML stack); CI installs `--with dev,ci`, and locally `make env` does the same.

The machine toolchain itself — `uv`, a supported Python, and `poetry` — is bootstrapped by `scripts/ensure_tools.sh`, which every `make` target reaches through `make tools`. `uv` is used as the installer for `poetry` (`uv tool install poetry`) and as the fallback Python version manager (`uv python install 3.13`); it is not used to resolve project dependencies — `poetry.lock` remains the single source of truth.

Design decisions & known limitations

- **Datasets are testable only through a model's `get_train_config()`.** A dataset needs `input_features` / `output_features` to be constructed, and those come from the model config — there is no self-describing `get_dummy_params()` on datasets. So a dataset with no model referencing it gets **discovered but not exercised**.
- **`SCRIPTAIDataset` is currently orphaned:** no model lists it in `supported_datasets`, so it appears in discovery (DATASET_ID `—`) but no pipeline/contract test runs on it. To cover it, add it to a model's `get_train_config()` (then it inherits fixtures, contract, and Zenodo tests for free). `M100Dataset` is likewise not wired to a model.
- **`importorskip` is banned in tests.** Every job runs the full `.install_ci` venv, so an `importorskip` could only fire on a broken environment — silently skipping and turning "nothing tested" into a green run. Import test dependencies directly so a missing dep fails loudly at collection.
- **No pair is excused from the pipeline, and several fail.** There is no xfail mechanism at all (see No expected failures), so `pytest --runxfail` produces exactly the same result as a normal run. `MACK-f-data` and `MCBound-f-data` fail because they target `output_features=['mem_compute_bound']`, a column absent from every F-DATA file — the published memory/compute-bound label is `pclass`. Two further problems surface once that is corrected: MACK reads `df.memory_bw` (the real column is `mbwidth`) and overwrites `df["pclass"]` with a tuple, which torch's `default_collate` rejects. All three are written up as `SUGGESTION` blocks inside the owning source files and tracked in issue #10 (closed). **CI reports this; CI does not fix it and does not hide it.**

No expected failures

`xfail` was removed from this repository on 2026-09-13. There is no `integration_xfail`, no `model_xfail`, no `config_load_xfail`, and no `pytest.mark.xfail` anywhere in `tests/`.

Why. A strict `xfail` marks a test that is *allowed* to fail: it still runs, the failure is still real, but pytest reports `XFAIL`, the exit code stays `0`, and the pipeline goes green. That is a green light over shipped code that does not work. The entries were honest and well documented — and the pipeline still said "pass" while three (model, dataset) pairs could not load their own data and one model could not be constructed at all. The signal was wrong, so the mechanism is gone.

What replaces it. Nothing suppresses a failure any more:

Situation	Before	Now
Model cannot be built	`model_xfail` → XFAIL, green	**FAIL**, red
Pair cannot load its fixture	`config_load_xfail` → XFAIL, green	**FAIL**, red
Pair cannot complete the lifecycle	`integration_xfail` → XFAIL, green	**FAIL**, red

The engineering record did not disappear with the lists: every one of those failures is registered under `core_change_requests` in `ci/policy.yml` — with the owner, the evidence, and a machine-checkable condition that **fails** the suite when the fix lands so the entry must be deleted. The register documents a defect; it does not silence the test.

Consequence, stated plainly. `main` is red until the model authors land those fixes. That is the intended state: `test:unit` and `test:integration` are hard gates, and the failures are

Failing	Owner	Registered as
`RUAD` — 8 cases (cannot be instantiated; `build_model()` builds a dataset in the validator)	RUAD model author	`CR-4`
`MACK-f-data`, `MCBound-f-data` (`output_features=['mem_compute_bound']` exists in no F-DATA file; the real column is `pclass`)	MACK / MCBound authors	`CR-5`
`JPCP-pm100` — 2 cases (`input_features` name columns the model's own preprocessing creates)	JPCP model author	`CR-6`

If you are tempted to bring `xfail` back: don't. Fix the code, or leave the test red and record the request. A suite whose green means "everything works except the parts we agreed to ignore" cannot be used as a gate.